Threat Intelligence Lab Guide

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation · Oct 23, 2026 · @Joshua Moses

Lab overview

By the end of this lab you will have built a working threat intelligence platform, pulled real indicators and reporting from open sources, turned that raw data into structured, enriched intelligence, profiled a threat actor, mapped its behavior to MITRE ATT&CK, and produced finished intelligence products for four different audiences. You will run the full cyber threat intelligence lifecycle, Direction through Feedback, on a single scenario. That is the point: threat intelligence is a process you practice, not a feed you subscribe to.

The scenario is simple and realistic. You work CTI for a mid-sized company. A peer organization in your sector was hit by a ransomware crew, and leadership wants to know whether you are exposed and what to do about it. Everything you build in this lab answers that question, which is exactly what effective intelligence does.

Who this is for. Students who have finished the virtualization lab (or who can already build and run a Linux VM) and are comfortable at a command line. No prior intelligence-analysis experience is needed. Every command and every analytic step is spelled out.

Time. Plan on 6 to 9 hours total. The build (Parts A through D) takes 2 to 3 hours and only has to be done once. The six exercises take 4 to 6 hours and can be split across sessions, since the platform persists.

What you need.

- A host machine with at least 16 GB RAM and 100 GB free disk. OpenCTI is memory-hungry; 24 to 32 GB on the host makes the lab comfortable.
- A hypervisor you already know (VMware Workstation, VirtualBox, Fusion, or UTM).
- Internet access from the lab VM. Unlike the incident response lab, this one needs the internet, because collecting open-source intelligence is the whole job.

Safety and legal rules, read these before you start.

1. Threat intelligence collection is legal, but how you collect matters. In this lab you use only passive, open-source techniques against targets you are allowed to query (your own domains, public threat feeds, published reporting). You do not scan, probe, or attack anyone.

2. OSINT tools like theHarvester query public records and search engines. Point them at your own organization's domain or a domain you have explicit permission to assess. Never run collection against a third party you have no relationship with.
3. You will handle real malware indicators (hashes, IPs, domains) from public feeds. Indicators are safe to store and analyze. Do not download or detonate actual malware samples as part of this lab.
4. Respect the sharing rules on everything you collect. Threat intelligence uses the Traffic Light Protocol (TLP) to mark what can be shared and with whom. You will learn to apply it, and you honor it: never redistribute something marked TLP:RED or TLP:AMBER outside its intended audience.
5. Snapshot the VM at the "clean build" stage before you start the exercises, so you can roll back and rerun the lab.

Principles of threat intelligence

Cyber Threat Intelligence (CTI) is the systematic collection, analysis, and interpretation of information about cyber threats, threat actors, their capabilities, intentions, and methods. Its purpose is to help an organization make informed security decisions and act before, during, and after an attack. The one principle under all the others: intelligence must be **actionable**, not just a pile of threat data. A feed of ten thousand IP addresses is data. Knowing that a specific actor who targets your sector is using three of those IPs, and what to do about it, is intelligence.

The 12 core principles. These are the ideas the exercises put into practice. Keep them in view as you work; each exercise calls back to several of them.

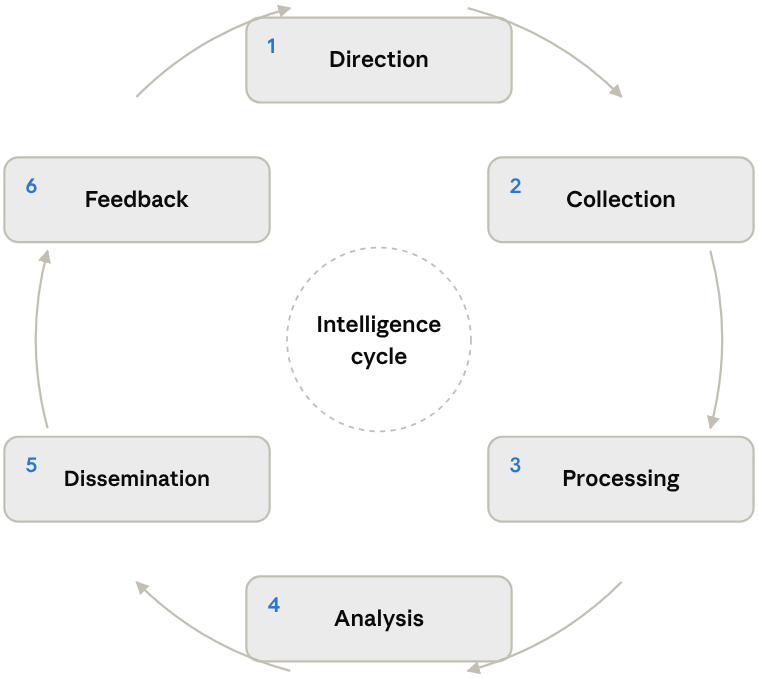
#	Principle	What it means in practice
1	Intelligence-driven security	Let analyzed threat information guide your defenses, instead of only reacting after something breaks
2	Intelligence requirements	Set Priority Intelligence Requirements (PIRs) from real business risk, critical assets, and likely adversaries
3	Collection and aggregation	Gather from OSINT, commercial feeds, internal telemetry, and incident reports, all lawfully

#	Principle	What it means in practice
4	Threat actor profiling	Understand an adversary's motivation, capability, targeting, and known behavior
5	Tactics, Techniques, and Procedures	Describe how an attacker operates, using a framework like MITRE ATT&CK
6	Indicators of Compromise	Analyze malicious IPs, domains, hashes, and URLs, while knowing their limits (they age fast)
7	Context and correlation	Connect data across sources to separate real threats from noise and false positives
8	Risk-based prioritization	Rank threats by relevance, exploitability, exposure, impact, and the defenses you already have
9	Timeliness and relevance	Deliver intelligence while it can still change a decision
10	Accuracy and confidence	Validate sources, separate fact from assessment, document uncertainty, assign confidence levels
11	Intelligence sharing	Share through trusted communities in standard formats like STIX and TAXII
12	Continuous improvement	Measure whether your intelligence helped, take feedback, and refine

The CTI lifecycle. The intelligence lifecycle is the continuous process that turns raw information into finished intelligence. The six exercises in this lab are the six stages of this cycle, in order.

Raw information becomes intelligence through a continuous cycle

Feedback from each cycle shapes the direction of the next



CTI lifecycle · 6 stages, continuous

The four levels of intelligence. The same incident produces different intelligence for different audiences. Knowing which level you are producing keeps you from handing a board a list of file hashes, or handing a SOC analyst a geopolitical essay.

Level	Focus	Primary audience
Strategic	Threat trends, geopolitics, business risk, adversary motivation	Executives, CISOs, boards
Operational	Campaigns, attacker objectives, infrastructure, anticipated activity	CTI teams, incident responders
Tactical	Adversary TTPs and the countermeasures for them	Threat hunters, security engineers
Technical	IOCs: domains, hashes, detection signatures	SOC analysts, SIEM and EDR platforms

Three foundational frameworks you will use directly in this lab:

- **MITRE ATT&CK** is a knowledge base of adversary tactics and techniques. You map observed behavior to it, build detections from it, and find your coverage gaps with it. You will use the ATT&CK Navigator to do exactly this.
- **The Cyber Kill Chain** describes the phases of an intrusion (reconnaissance through actions on objectives) and helps you find where to break the attack before the adversary reaches its goal.
- **The Diamond Model of Intrusion Analysis** connects four features of any malicious event: adversary, capability, infrastructure, and victim. It is the structure you will use to profile the actor in the Analysis exercise.

What makes CTI effective. A good CTI program answers five questions. Every product you build in this lab should answer at least one of them, clearly.

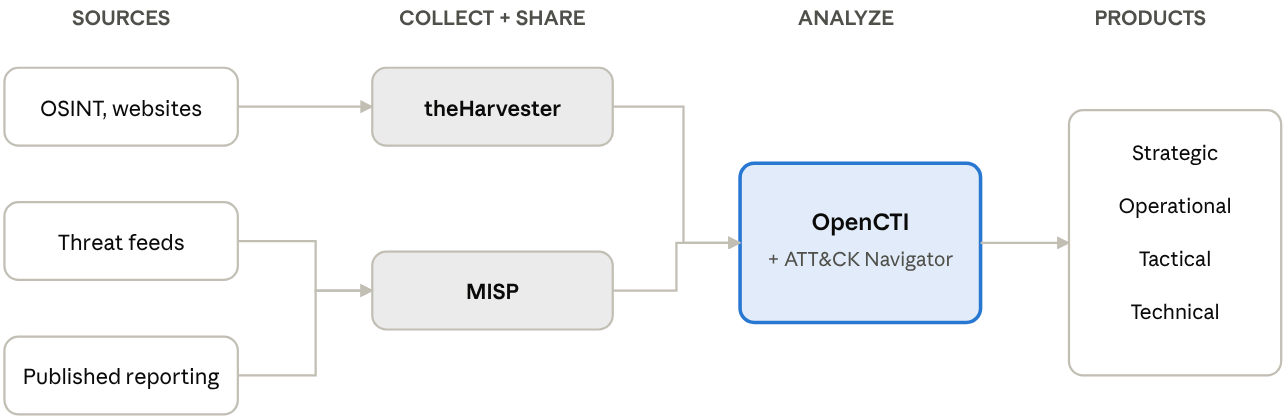
1. Who is targeting the organization?
2. Why are they targeting it?
3. How are they likely to attack?
4. What assets, identities, systems, or information are at risk?
5. What should defenders do about it?

The difference between data and intelligence lives in that last question. Reporting that "attackers are exploiting a vulnerability" is data. Intelligence tells you whether you run the affected software, whether it is being exploited in the wild, what evidence to hunt for, and which mitigation to apply first.

Lab architecture and software

Unlike the incident response lab, this one runs on a single Linux VM. The tools are containers and command-line utilities, not a multi-machine network. One capable VM holds the whole CTI stack.

Sources flow through collection into the TIP, out as products



lab data flow · sources to products

Component	Role	Delivery	Access
OpenCTI	Threat intelligence platform (the TIP): stores, links, and visualizes all intelligence	Docker Compose stack	Web UI on port 8080
MISP	Threat intelligence sharing platform: feeds, events, IOC sharing via STIX/TAXII	Docker Compose stack	Web UI on port 443
ATT&CK Navigator	Visualize and annotate adversary TTPs on the ATT&CK matrix	Hosted, or local Docker	Web UI
theHarvester	OSINT collection: subdomains, emails, hosts from public sources	Python CLI	Terminal

The VM: Ubuntu Server 22.04 or 24.04, 8 vCPU, 16 GB RAM, 80 GB disk. OpenCTI is the heavy piece; give it as much RAM as you can spare. If your host cannot run OpenCTI and MISP at once, the guide notes where you can run them one at a time.

The software, and why each tool is here.

- **OpenCTI** is your intelligence platform, the center of the lab. It is where raw data becomes connected intelligence: you ingest indicators, reports, and actors, and OpenCTI links them into a graph you can query and visualize. It speaks STIX natively, so it is the hub that the other tools feed. This is where you will do most of the Processing and Analysis work. Free and open source, and widely used in real CTI teams.

- **MISP** (Malware Information Sharing Platform) is the sharing and feed-ingestion workhorse. Where OpenCTI is your internal brain, MISP is how intelligence flows in and out: it pulls from dozens of community threat feeds and publishes your own findings in standard formats. It demonstrates principle 11, intelligence sharing, concretely. This lab uses MISP 2.5.
- **MITRE ATT&CK Navigator** is how you turn actor behavior into a visual coverage map. You will build a layer showing the techniques your scenario actor uses, then compare it to what you can detect, which surfaces your gaps. It is the practical tool for principle 5 (TTPs).
- **theHarvester** is a passive OSINT collector. It queries public search engines and records to find the subdomains, email addresses, and hosts tied to a domain, which is how you see your own externally visible attack surface the way an adversary would. It is your hands-on tool for the Collection stage.

Everything is free and open source. The only accounts you may want are free API keys for a few OSINT sources (for example Shodan or VirusTotal), which make collection richer but are not required.

Part A: build the lab VM and install Docker

One VM holds everything. If you did the virtualization lab, the VM creation is familiar; the new work is installing Docker, which both OpenCTI and MISP run on.

A1. Create the VM. Build a VM with 8 vCPU, 16 GB RAM, 80 GB disk and install Ubuntu Server 22.04 or 24.04. Give it internet access (NAT is fine; this lab needs the internet). Update it:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg git jq
```

A2. Install Docker Engine and Compose. Add Docker's official repository and install the engine plus the Compose plugin:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

A3. Let your user run Docker without sudo.

```
sudo usermod -aG docker $USER
newgrp docker
docker --version
docker compose version
```

Both version commands should print cleanly. Docker 27+ and Compose v2.29+ are what this lab expects.

A4. Raise the kernel map-count limit. OpenCTI's search engine (Elasticsearch/OpenSearch) needs this or it will not start. Set it now and make it permanent:

```
sudo sysctl -w vm.max_map_count=1048575
echo 'vm.max_map_count=1048575' | sudo tee -a /etc/sysctl.conf
```

A5. Note the VM's IP. Run `ip a` and record the address. You will reach the web UIs from your host browser at this IP. For the rest of the guide, `LAB_IP` means this address.

That is the whole foundation. Everything else is deploying tools onto it.

Part B: deploy and configure OpenCTI (the TIP)

OpenCTI ships as a Docker Compose stack: the platform, a worker, a graph database, a search engine, Redis, RabbitMQ, and an S3-compatible store. The provided compose file wires them together; you supply a few environment values. Reference: OpenCTI installation.

B1. Clone the Docker helper repo.

```
mkdir -p ~/opencti && cd ~/opencti
git clone https://github.com/OpenCTI-Platform/docker.git
cd docker
```

B2. Generate the environment file. OpenCTI needs several unique tokens and an admin login. This one-liner writes a working `.env` with fresh UUIDs; edit the admin email and password afterward:

```
cat << EOF > .env
OPENCTI_ADMIN_EMAIL=admin@cti.lab
OPENCTI_ADMIN_PASSWORD=ChangeMeStrong1
OPENCTI_ADMIN_TOKEN=$(cat /proc/sys/kernel/random/uuid)
OPENCTI_BASE_URL=http://LAB_IP:8080
OPENCTI_HEALTHCHECK_ACCESS_KEY=$(cat /proc/sys/kernel/random/uuid)
MINIO_ROOT_USER=$(cat /proc/sys/kernel/random/uuid)
MINIO_ROOT_PASSWORD=$(cat /proc/sys/kernel/random/uuid)
RABBITMQ_DEFAULT_USER=guest
RABBITMQ_DEFAULT_PASS=guest
ELASTIC_MEMORY_SIZE=4G
CONNECTOR_HISTORY_ID=$(cat /proc/sys/kernel/random/uuid)
CONNECTOR_EXPORT_FILE_STIX_ID=$(cat /proc/sys/kernel/random/uuid)
CONNECTOR_EXPORT_FILE_CSV_ID=$(cat /proc/sys/kernel/random/uuid)
CONNECTOR_IMPORT_FILE_STIX_ID=$(cat /proc/sys/kernel/random/uuid)
CONNECTOR_IMPORT_DOCUMENT_ID=$(cat /proc/sys/kernel/random/uuid)
SMTP_HOSTNAME=localhost
EOF
```

Edit `.env` and replace `LAB_IP` with your VM's actual IP, and set a real admin password. Keep the `OPENCTI_ADMIN_TOKEN` value; you will need it to connect MISP and the OSINT tools later.

B3. Start the stack. The first run pulls several gigabytes of images and takes 10 to 20 minutes to become healthy.

```
docker compose up -d
```

B4. Watch it come up.

```
docker compose ps
docker compose logs -f opencti
```

Wait until the opencti container reports it is listening. The search engine takes the longest; be patient on the first boot.

B5. Log in. From your host browser, go to `http://LAB_IP:8080`. Log in with the admin email and password from your `.env`. You land on an empty OpenCTI dashboard. This is your intelligence platform, ready for data.

B6. Get your API token. In the UI, open your profile (top right) and find your API access token (it matches `OPENCTI_ADMIN_TOKEN`). Copy it. You will paste it into MISP's connector and into theHarvester's export step so those tools can push data into OpenCTI.

B7. Enable a couple of built-in connectors (optional but recommended). OpenCTI connectors ingest reference data. From the compose file you can enable the MITRE ATT&CK connector, which loads the full ATT&CK dataset into your platform so actors and techniques resolve to real objects. If you enable it, re-run `docker compose up -d`. This makes the Analysis exercise much richer.

If OpenCTI will not start, the usual cause is the map-count limit (step A4) or not enough RAM. See Troubleshooting.

Part C: deploy and configure MISP

MISP is your feed-ingestion and sharing platform. The official misp-docker project is the fastest way to a working instance. Reference: misp-docker.

Memory note. OpenCTI and MISP together want more than 16 GB. If your VM is tight, run them one at a time: `docker compose stop` in the OpenCTI folder before bringing MISP up, and swap back when you need OpenCTI. The exercises tell you which tool you need when.

C1. Clone and configure.

```
cd ~ && git clone https://github.com/MISP/misp-docker.git
cd misp-docker
cp template.env .env
```

Edit `.env` and set at least these:

- `BASE_URL=https://LAB_IP` (your VM's IP)
- `ADMIN_EMAIL=admin@cti.lab`
- `ADMIN_PASSWORD=` a strong password you choose

C2. Pull and start. The images are prebuilt, so no local build is needed.

```
docker compose pull
docker compose up -d
```

The first boot initializes the database and can take several minutes. Watch with `docker compose logs -f misp-core`.

C3. Log in. Browse to `https://LAB_IP` (HTTPS, self-signed certificate, accept the warning). Log in with the admin email and password from `.env`. On first login MISP forces a password change; set your real one.

C4. Turn on threat feeds. This is the point of MISP. In the UI go to Sync Actions > List Feeds. MISP ships with a large set of community feeds predefined (CIRCL, abuse.ch, and others). Enable a few well-known ones (for example the CIRCL OSINT feed and an abuse.ch feed), then select them and choose Fetch and store all feed data. Within a few minutes MISP populates with real events and indicators from the wider community. You now have live threat data flowing in, which is principle 3 (collection and aggregation) and principle 11 (sharing) in action.

C5. Create an API key. Go to Administration > List Auth Keys > Add authentication key for your admin user. Copy the key. You will use it to connect MISP to OpenCTI.

C6. (Optional) Connect MISP to OpenCTI. OpenCTI has a MISP connector that pulls MISP events into your TIP automatically, so indicators collected in MISP become analyzable objects in OpenCTI. In the OpenCTI docker folder, add the MISP connector service to the compose file with your MISP URL (`https://LAB_IP`), the MISP API key from C5, and your OpenCTI token, then `docker compose up -d`. This bridge is what makes the two platforms one pipeline. If you are running the tools one at a time, skip this and import manually in the Processing exercise instead.

If the MISP UI shows database or worker errors on first boot, give it more time or restart the stack; see Troubleshooting.

Part D: ATT&CK Navigator and OSINT tools

Two lighter tools finish the stack: the ATT&CK Navigator for visualizing TTPs, and theHarvester for OSINT collection.

D1. ATT&CK Navigator. The Navigator is a client-side web app. You have two options:

- **Hosted (simplest):** use the official instance at mitre-attack.github.io/attack-navigator. Layer files you create are not uploaded anywhere; the app runs entirely in your browser. This is fine for the lab, since your scenario uses public ATT&CK techniques,

not sensitive data.

- **Self-hosted (if you want it local):** run a container on the lab VM.

  ```
  docker run -d --name attack-navigator -p 8081:80 fgierlinger/mitre-attack-navigator:latest
  ```

  Then browse to `http://LAB_IP:8081`. Host your own instance any time your layers would contain sensitive internal data.

Confirm it loads the Enterprise matrix. That is all the setup it needs.

D2. theHarvester. Install it on the lab VM. The cleanest way is a Python virtual environment:

```
sudo apt install -y python3-pip python3-venv
cd ~ && python3 -m venv harvester-env
source harvester-env/bin/activate
pip install theHarvester
theHarvester --help
```

If the pip package gives you trouble, install from source instead:

```
git clone https://github.com/laramies/theHarvester.git
cd theHarvester
python3 -m pip install -r requirements.txt
python3 theHarvester.py --help
```

D3. (Optional) Add free API keys. theHarvester works with no keys against several sources, but adding free keys (Shodan, VirusTotal, and others) widens its reach. Create an API key file at `~/.theHarvester/api-keys.yaml` and add keys you have. Skip this if you want; the lab works without them.

D4. Snapshot the clean build. With Docker installed, OpenCTI and MISP deployed and reachable, the Navigator working, and theHarvester installed, power down and take a VM snapshot named `clean-build`. Roll back to it any time you want to redo the lab or recover from a mistake.

The build is done. Everything from here is threat intelligence work.

Exercise 1: Direction

Stage goal: define what the organization needs to know and why. Intelligence without direction is just data collection. This stage sets the questions every later stage must answer. Principles in play: intelligence requirements (2), risk-based prioritization (8).

The scenario, restated. A peer company in your sector was hit by a ransomware group. Leadership asks: are we exposed, and what should we do? Your job in this stage is to turn that vague worry into specific, answerable intelligence requirements.

What you will do

1. **Write your Priority Intelligence Requirements (PIRs).** Using the worksheet in the Appendix, write three to five PIRs that this scenario demands. Good PIRs are specific and decision-focused. For example: "Which ransomware groups are currently targeting our sector?" "What initial-access techniques do they use?" "Do we expose any of the services they exploit?" "What indicators should our SOC be hunting for right now?" Each PIR should map to one of the five effectiveness questions.
2. **Identify your critical assets and likely adversaries.** List what you are protecting (the crown jewels) and who plausibly threatens it. You cannot prioritize threats without knowing what matters and who is likely to come after it.
3. **Define collection priorities.** For each PIR, note where the answer is likely to live: a public threat feed, published vendor reporting, your own externally visible attack surface. This becomes your collection plan for Exercise 2.
4. **Set confidence and timeliness expectations.** Note how fresh the intelligence needs to be to matter. An indicator feed answer is useful for days; a strategic trend answer is useful for months. Writing this down now keeps you from over-investing in data that will be stale before it is used.

Intended result and why. You leave this stage with a short, written set of PIRs tied to real business risk, a critical-asset list, likely adversaries, and a collection plan. This matters because direction is the control surface for the entire cycle. Teams that skip it collect everything and conclude nothing, because they never decided what question they were answering. Every later exercise checks its output against these PIRs, which is how you keep the work actionable instead of academic.

Deliverable: a completed PIR worksheet with three to five prioritized, decision-focused requirements and a collection plan.

Exercise 2: Collection

Stage goal: gather relevant information from the sources your collection plan named, lawfully. Principles in play: collection and aggregation (3), indicators of compromise (6).

Pick your scenario actor. To make this concrete, choose a well-documented ransomware group with public reporting, for example LockBit, BlackCat/ALPHV, or another with abundant open-source coverage. You will collect everything you can about it, then narrow to what answers your PIRs.

What you will do

1. **Collect published reporting (OSINT).** Find two or three solid open-source reports on your chosen actor: a vendor threat report, a CISA advisory, the MITRE ATT&CK group page. Note the URL, date, and source for each. Freshness and source quality matter; a two-year-old report describes an actor that has probably changed.
2. **Pull indicators from MISP feeds.** Bring up MISP. Search its ingested events for your actor's name or associated malware family. Export the matching indicators (IPs, domains, hashes) as a STIX or CSV file. These are your technical-level artifacts.
3. **Map your own attack surface with theHarvester.** This is where you see yourself the way an adversary does. Run theHarvester against a domain you own or are authorized to assess:

   ```
   source ~/harvester-env/bin/activate
   theHarvester -d yourdomain.com -b bing,crtsh,duckduckgo -f harvest-
   yourdomain
   ```

   This returns subdomains, hosts, and email addresses visible in public sources. Review what is exposed. If your scenario actor's initial-access technique is phishing, that harvested email list is exactly what they would target.
4. **Record provenance for everything.** For each item collected, note the source, the date, and a first-pass reliability judgment. This is the raw material for the accuracy and confidence work later. Collection without provenance cannot be assessed for confidence.

Intended result and why. You leave this stage with a collection folder: published reports with citations, an indicator export from MISP, and a theHarvester report of your own exposure, each tagged with source and date. This matters because the quality of everything downstream is capped by the quality of collection. Garbage in, garbage out is literally true for intelligence. The discipline of recording provenance now is what lets you assign honest confidence levels in Analysis, rather than guessing.

Deliverable: a documented collection set (reports, indicators, exposure data) with source and date recorded for every item.

Exercise 3: Processing

Stage goal: normalize, enrich, deduplicate, and organize the collected data into a structured form you can analyze. Raw collection is messy: different formats, duplicates, no connections. Processing fixes that. Principles in play: collection and aggregation (3), context and correlation (7).

This is where OpenCTI earns its place. Bring up OpenCTI. You will load your collected data into it so that scattered items become connected, queryable objects.

What you will do

1. **Import the indicators.** Load your MISP indicator export into OpenCTI. If you built the MISP connector in Part C, the events are already flowing in; confirm they appear under Observations and Indicators. If not, use OpenCTI's Data Import to upload the STIX or CSV file from Exercise 2.
2. **Create the actor and report objects.** In OpenCTI, create a Threat Actor (or Intrusion Set) object for your scenario group, and create Report objects for each published report you collected, with their real dates and sources. This turns a pile of PDFs into structured intelligence entities.
3. **Link everything.** Connect the indicators to the actor, the reports to the actor, and the malware families to the actor. OpenCTI's whole value is the graph: once linked, you can see at a glance what infrastructure, tooling, and reporting tie to this adversary. This is context and correlation made literal.
4. **Deduplicate and normalize.** OpenCTI merges duplicate observables automatically, but review for near-duplicates and inconsistent naming (the same actor under two aliases, the same domain typed two ways). Clean data is a precondition for trustworthy analysis.
5. **Enrich.** Use OpenCTI's enrichment on a few key indicators (for example, resolve a domain, or pull reputation). Enrichment adds the context that turns a bare indicator into something an analyst can reason about. Note which enrichments added real value and which did not; not all enrichment is worth the noise.

Intended result and why. You leave this stage with a populated OpenCTI knowledge graph: your actor at the center, linked to its indicators, malware, and the reports that document it, deduplicated and enriched. This matters because analysis is only as good as

the structure under it. You cannot see a campaign in a spreadsheet of unconnected rows; you can see it instantly in a graph. Processing is the unglamorous step that makes the Analysis insight possible.

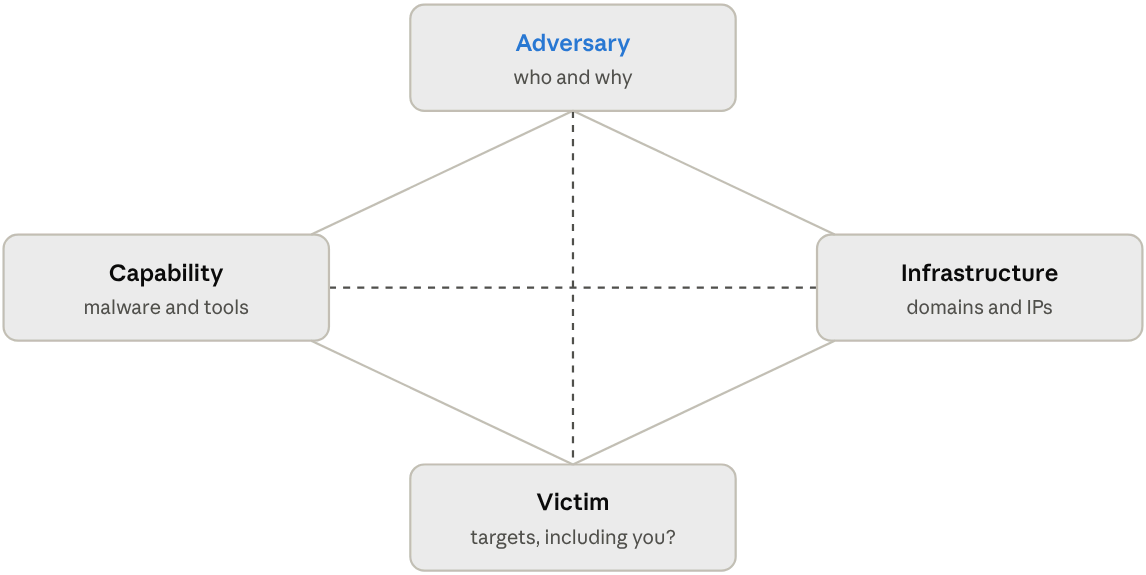
Deliverable: a populated OpenCTI graph centered on your actor, with indicators, reports, and malware linked, deduplicated, and enriched.

Exercise 4: Analysis

Stage goal: evaluate the threat, establish context, assess confidence, and develop findings. This is the heart of intelligence work: turning organized data into judgments a decision-maker can act on. Principles in play: threat actor profiling (4), TTPs (5), context and correlation (7), accuracy and confidence (10).

The Diamond Model: four features of every intrusion

Profile the actor by connecting all four vertices, not listing them



Diamond Model · 4 vertices

What you will do

1. **Profile the actor with the Diamond Model.** Using the template in the Appendix, fill in the four vertices for your scenario actor: adversary (who they are, their motivation), capability (their malware and tools), infrastructure (the domains and IPs they use), and

victim (who they target, and whether that includes you). The Diamond Model forces you to connect these four, which is how you reason about an intrusion rather than just list facts.

2. **Map the TTPs in ATT&CK Navigator.** Open the Navigator, create a new Enterprise layer, and highlight every technique your actor is reported to use (from the reports you collected and the ATT&CK group page). Give the layer a clear name and score the techniques. You now have a visual fingerprint of how this actor operates.
3. **Compare to your defenses (gap analysis).** Create a second Navigator layer marking the techniques you can currently detect. Combine the two layers: the techniques the actor uses that you cannot detect are your gaps, and they are the most valuable output of this whole lab. These gaps answer PIR-style questions directly.
4. **Correlate against your own exposure.** Cross-reference the actor's initial-access TTPs against the attack surface you mapped with theHarvester in Exercise 2. If the actor favors phishing and you found a large exposed email footprint, that is a specific, evidenced risk, not a generic worry.
5. **Assign confidence to every finding.** For each judgment, separate fact from assessment and assign a confidence level (high, moderate, low) with a one-line reason, using the confidence template in the Appendix. "The actor uses technique X" from three independent vendor reports is high confidence. "The actor is likely to target us next" is an assessment, probably moderate or low, and you say so. Honest confidence is what separates intelligence from speculation.

Intended result and why. You leave this stage with a completed Diamond Model profile, an ATT&CK layer of the actor's TTPs, a gap analysis against your detections, a correlation to your real exposure, and confidence-rated findings. This matters because this is the stage that produces actual intelligence: not "here is what we found" but "here is what it means, how sure we are, and where you are exposed." The gap analysis and the exposure correlation are the findings leadership actually asked for.

Deliverable: a Diamond Model profile, two ATT&CK Navigator layers (actor TTPs and your coverage), a gap list, and a set of confidence-rated findings.

Exercise 5: Dissemination

Stage goal: deliver intelligence to the right audience in a usable format. The best analysis is worthless if it reaches the wrong person in the wrong form. Principles in play: timeliness and relevance (9), intelligence sharing (11).

The core skill here: the same findings become four different products for the four levels of intelligence. You will produce at least two, and sketch the others.

What you will do

1. **Write the strategic product (for leadership).** One page, plain language, no jargon. Answer the question leadership actually asked: are we exposed to this threat, and what are we doing about it? Frame it as business risk, not technical detail. Use the report template in the Appendix. This is what the CISO reads.
2. **Write the operational/tactical product (for the security team).** This is the actor profile: the Diamond Model, the ATT&CK TTP layer, and the prioritized gap list from Exercise 4. It tells threat hunters and engineers how the actor operates and what to build or hunt for. Include the specific detection gaps as recommended actions.
3. **Package the technical product (for the SOC and tools).** Export the indicators from OpenCTI as a STIX 2.1 bundle. This is the machine-readable product: a block list and detection feed the SOC can load into a SIEM or EDR. From OpenCTI, use the export to produce a clean STIX file.
4. **Share through MISP (STIX/TAXII).** Publish your findings as a MISP event with proper Traffic Light Protocol marking. Apply TLP tags correctly: TLP:CLEAR for what can be shared publicly, TLP:GREEN or TLP:AMBER for community-only. This is principle 11 made real, and it demonstrates the standard formats (STIX, TAXII) the industry shares in. Confirm the event is retrievable via MISP's API, which is how a TAXII consumer would pull it.
5. **Match format to audience, deliberately.** Before you finish, check each product against its audience: would an executive understand the strategic one without a security background? Could a SOC tool actually ingest the technical one? A product delivered in the wrong format for its reader has not been disseminated, it has just been produced.

Intended result and why. You leave this stage with a one-page strategic brief, a technical actor profile with gap analysis, a STIX indicator bundle, and a TLP-marked MISP event others could consume. This matters because dissemination is where intelligence becomes action, or dies in a drafts folder. The discipline of producing for a specific audience, in a format they can use, on a timeline where it still matters, is what makes a CTI function valuable rather than ignored. The TLP marking is not bureaucracy; mishandling it is how trust in sharing communities gets destroyed.

Deliverable: a strategic brief, a technical/tactical actor profile, a STIX 2.1 export, and a TLP-marked MISP event.

Exercise 6: Feedback

Stage goal: determine whether the intelligence answered the original questions and improved decisions, then refine the next cycle. This is the stage that makes a CTI program get better instead of just busier. Principles in play: continuous improvement (12), accuracy and confidence (10).

What you will do

1. **Grade yourself against your PIRs.** Go back to the Priority Intelligence Requirements you wrote in Exercise 1. For each one, did your products actually answer it? Mark each: fully answered, partially answered, or not answered. An honest no here is more valuable than a dishonest yes, because it tells you where to collect next.
2. **Assess the collection.** Which sources gave you the most useful material, and which were noise? A feed that produced nothing relevant is a feed to drop; a report that answered three PIRs is a source to prioritize. This refines your collection plan for the next cycle.
3. **Check your confidence calibration.** Review the confidence levels you assigned in Analysis. Were any findings that you called high confidence actually shaky? Were any low-confidence judgments better supported than you thought? Calibration improves only when you review it.
4. **Identify gaps and next requirements.** What did this cycle reveal that you now need to know? Detection gaps become engineering tickets; unanswered PIRs become collection priorities; a newly relevant actor becomes a new requirement. These feed directly back into Direction.
5. **Measure the program.** Note simple metrics: how long from PIR to delivered product, how many PIRs were answered, how many detection gaps you found. You cannot improve what you do not measure, and these numbers are how you show leadership the CTI function earns its keep.
6. **Write the refinement note.** One short page: what worked, what did not, what changes for next cycle. This is the artifact that closes the loop and seeds the next Direction stage.

Intended result and why. You leave this stage with a PIR scorecard, a source-quality assessment, a confidence-calibration review, a list of new requirements, program metrics, and a refinement note. This matters because the intelligence lifecycle is a loop, and feedback is what makes the next turn of the loop better than the last. A CTI program

without feedback keeps answering yesterday's questions with yesterday's sources. The teams that do feedback well get measurably sharper over time; the ones that skip it plateau.

Deliverable: a PIR scorecard, a source assessment, a confidence-calibration review, new requirements for the next cycle, and a one-page refinement note.

Expected results, deliverables, and grading

By the end of the lab a student should be able to show a populated TIP, an actor profile mapped to ATT&CK, intelligence products for different audiences, and a full cycle that closes with feedback. The table below is both a student checklist and a grading rubric.

Stage	Deliverable	What "done" looks like
Direction	PIR worksheet	Three to five specific, decision-focused PIRs tied to real risk, with a collection plan
Collection	Documented collection set	Published reports, MISP indicator export, theHarvester exposure data, each with source and date
Processing	Populated OpenCTI graph	Actor linked to indicators, reports, and malware; deduplicated and enriched
Analysis	Profile + TTP map + gaps	Diamond Model complete, two ATT&CK layers, a gap list, confidence-rated findings
Dissemination	Audience products	Strategic brief, tactical profile, STIX 2.1 export, TLP-marked MISP event
Feedback	Cycle review	PIR scorecard, source assessment, confidence review, new requirements, metrics, refinement note

Grading suggestion (100 points). Each of the six stages is worth up to 14 points for completeness and correctness of its deliverable, for 84 total. The remaining 16 points: 8 for the quality of the strategic brief (can a non-technical leader act on it?) and 8 for correct TLP handling and a valid STIX export (does the sharing actually work?). Those two prove the student can both communicate to decision-makers and operate in the shared ecosystem, which is what a CTI analyst is for.

Minimum pass: a student passes when they produced written PIRs, collected real open-source material with provenance, built an actor profile in OpenCTI mapped to ATT&CK, and delivered at least one audience-appropriate product. As in real CTI, the judgment and the writing matter as much as the tooling. An analyst who can run OpenCTI but cannot answer "so what should we do?" has not done the job.

Troubleshooting

Symptom	Likely cause	Fix
OpenCTI never becomes healthy	Search engine failed to start	Confirm `vm.max_map_count` is set (step A4); confirm the VM has enough free RAM; check `docker compose logs elasticsearch`
OpenCTI UI loads but is empty of ATT&CK data	Reference connectors not enabled	Enable the MITRE connector in the compose file and re-run `docker compose up -d`
Containers restart in a loop	Not enough memory	Lower `ELASTIC_MEMORY_SIZE` in `.env`, or run OpenCTI and MISP one at a time
MISP shows database errors on first boot	Database still initializing	Wait a few minutes; if it persists, `docker compose down` then `up -d` again
MISP feeds fetch nothing	Feeds enabled but not fetched	In List Feeds, select the feeds and choose Fetch and store all feed data; confirm the VM has internet
Cannot reach a web UI from the host	Wrong IP or firewall	Confirm `LAB_IP` with `ip a`; confirm the port (8080 OpenCTI, 443 MISP); check the host and VM firewalls
MISP-to-OpenCTI connector pulls nothing	Wrong URL, key, or token	Recheck the MISP URL, the MISP API key, and the OpenCTI token in the connector config; check the connector container logs

Symptom	Likely cause	Fix
theHarvester returns almost nothing	No API keys, or rate-limited	Try different `-b` sources; add free API keys (step D3); some sources rate-limit, so wait and retry
STIX export is empty	Nothing linked to the exported object	Confirm the actor and indicators are linked in OpenCTI before exporting

If a tool gets into a bad state, that is what the `clean-build` snapshot is for. Roll back and redo the affected part.

Appendix: templates

Copy these into your own notes and fill them in as you work.

PIR worksheet (Direction)

PIR #	The question	Why it matters (business risk)	Where the answer lives	Effectiveness question it maps to

Diamond Model profile (Analysis)

Vertex	Entry
Adversary (who, motivation)	
Capability (malware, tools)	
Infrastructure (domains, IPs)	
Victim (targeting, your exposure)	

Confidence log (Analysis)

Finding	Fact or assessment?	Sources	Confidence (high/mod/low)	One-line reason

Intelligence report template (Dissemination)

Use this shape for each audience product. Keep the strategic version to one page in plain language.

Section	Content
Title and date	
Audience / level	
TLP marking	
Bottom line up front (BLUF)	
Key findings (with confidence)	
Who / why / how / what is at risk	
Recommended actions (prioritized)	
Indicators (technical products only)	
Sources	

PIR scorecard (Feedback)

PIR #	Answered? (full/partial/no)	Which product answered it	What to collect next cycle

References

- OpenCTI documentation and installation
- MISP project and misp-docker
- MITRE ATT&CK and the ATT&CK Navigator
- theHarvester, laramies
- The Diamond Model of Intrusion Analysis (Caltagirone, Pendergast, Betz)
- Lockheed Martin Cyber Kill Chain
- STIX and TAXII, OASIS
- Traffic Light Protocol (TLP), FIRST

Software versions referenced in this guide: OpenCTI (current Docker release), MISP 2.5. Check each project's release page for the current version before you build; the commands change little, but always pull the latest stable release.

Cybersecurity Masterclass by Joshua Moses | CyberTrace Foundation.